



Actividad de recuperación día 5

CSS GRID, FLEXBOX, POSITION: STICKY Y TARJETAS

AUTO-FIT

Nombre del aprendiz	Sneider Alvarez Santos	Número de ficha	3428392
Documento	1016717046	Fecha de inicio	15/09/2026
Trimestre	tercero	Fecha de entrega	Lunes 28 de septiembre de 2026 · 5:00 p. m.
Medio de entrega	Microsoft Teams — tarea asignada al aprendiz en el equipo del programa. No se reciben entregas por correo, memoria USB ni mensajería instantánea.		
Competencia	Desarrollar el sistema de información que cumpla con los requisitos de la solución informática.		
Resultados de aprendizaje por recuperar	Construir interfaces de usuario aplicando principios de diseño visual y de accesibilidad. Codificar módulos del sistema con control de acceso, persistencia de datos y generación de reportes.		
Causal del plan de mejoramiento	Con la identidad visual y las tres páginas ya estilizadas, faltaba la maquetación del tablero. Sin CSS Grid el tablero no tiene estructura de dos columnas, el menú lateral no tiene un ancho fijo, la barra superior se va al bajar por la página y las cuatro tarjetas de indicadores no se reacomodan solas al angostar la ventana.		

Bitácora del día: qué hice, qué error encontré y cómo lo resolví

Qué hice

- Maqueté `dashboard.html` completo con **CSS Grid**: declaré `grid-template-areas` para nombrar las cuatro zonas — barra, menú, contenido y pie — sin usar `float` ni `position: absolute`.
- Dejé el menú lateral con **240 px fijos** y la columna del contenido ocupando el resto con `minmax(0, 1fr)`, para que la tabla ancha no pueda sacar el menú de la pantalla.
- Creé la fila de **cuatro tarjetas de indicadores** con `repeat(auto-fit, minmax(220px, 1fr))` y verifiqué que se reacomodan solas al angostar la ventana.
- Fijé la barra superior con `position: sticky` y comprobé el desplazamiento con la tabla larga de 16 mesas.
- Medí el resultado en el navegador de verdad: ancho real del menú, desbordamiento horizontal y reacomodación de las tarjetas en cinco anchos distintos.
- Escribí en el punto 4 de esta bitácora en qué caso usé Grid y en cuál Flexbox, y por qué.

Qué error encontré y cómo lo resolví

Error encontrado	Cómo se detectó	Cómo lo resolví
La tabla se salía de la pantalla en ventanas angostas. A 520 px de ancho la tabla llegaba hasta el píxel 519, cuando el límite del área de contenido era el 508: se pasaba 11 px.	Renderizando la página a cinco anchos y midiendo, sobre el píxel ya pintado, dónde terminaba el fondo verde del encabezado de la tabla.	En vez de deixar que la tabla rompa la maquetación, la metí en un contenedor con <code>overflow-x: auto</code> . Así la tabla se desliza sola por dentro y la rejilla no se entera. Volví a medir y la tabla quedó en 507, dentro del límite.

Error encontrado	Cómo se detectó	Cómo lo resolví
Escribir la columna como <code>1fr</code> en vez de <code>minmax(0, 1fr)</code>. Un <code>1fr</code> tiene un mínimo automático igual a <code>min-content</code> , o sea que la columna nunca puede quedar más angosta que el contenido que tiene dentro.	Al notar que la tabla, que es ancha, estaba intentando ensanchar la columna del contenido en vez de acomodarse dentro de ella.	Cambié a <code>grid-template-columns: 240px minmax(0, 1fr)</code> . Con el <code>0</code> como mínimo, la columna sí puede encogerse por debajo de la tabla y el menú queda siempre en sus 240 px.
La guía usa variables que en nuestro proyecto no existen. Escribe <code>var(--borde)</code> , y en el menú trae un <code>#15231A</code> suelto.	Comparando el enunciado con las variables realmente definidas en <code>css/tokens.css</code> y <code>css/estilos.css</code> antes de escribir la regla.	Mapé <code>var(--borde)</code> a <code>var(--border-color)</code> y ese hexadecimal a <code>var(--color-marca-hover)</code> , que es el mismo verde del menú. Con eso la hoja queda sin ningún hexadecimal suelto, como piden los criterios.
No podía leer las medidas del navegador. El comando <code>--dump-dom</code> de Edge salió con <code>0</code> líneas: esa opción ya no existe en la versión 153 del navegador.	Intentando automatizar la medición de las cuatro zonas de la rejilla.	Cambié el método: el script de medición escribe cada dato dentro del color de un bloque de la pantalla, y yo leo ese color por píxel. Es un canal tonto pero funciona y deja el dato en un lugar verificable.
La captura automática salía en blanco si la página queda desplazada. Así que no se podía "ver" la barra pegada arriba después de bajar.	Al intentar capturar la página con <code>scrollTo</code> aplicado: la imagen salió con el fondo plano, sin ningún contenido.	Medí primero, vuelvo a subir al inicio y después pinto el resultado. Así la medida sale de la posición real de la barra, no de mirar una foto.

Punto 1 — Maquetar `dashboard.html` con Grid

La página se reparte en cuatro zonas: la barra superior, el menú lateral izquierdo, el contenido y el pie. En vez de colocar cada bloque por coordenadas, le puse nombre a cada zona con `grid-template-areas`, y así el HTML decide dónde va cada bloque con `grid-area`. Si algún día hay que mover algo, se cambia una línea de la plantilla y no el `margin` de cuatro elementos.

```
.panel {
  display: grid;
  grid-template-columns: 240px minmax(0, 1fr);
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "barra barra"
    "menu contenido"
    "pie pie";
  min-height: 100vh;
}

.panel__barra { grid-area: barra; }
.panel__menu { grid-area: menu; }
.panel__contenido { grid-area: contenido; }
.panel__pie { grid-area: pie; }
```

El menú se queda en **240 px fijos** y la columna del contenido se queda con *todo* el espacio que sobre. Para el menú usé Flexbox en la lista de enlaces, que es una sola columna de elementos y Flexbox lo resuelve en una línea.

La diferencia entre `1fr` y `minmax(0, 1fr)` no es un detalle de escritura. Un `1fr` nunca baja del ancho de su contenido, así que la tabla de 16 mesas habría empujado la columna y el menú se habría salido de la pantalla. Con `minmax(0, 1fr)` la columna sí puede quedar más angosta que la tabla.

Medición del menú y del desbordamiento

Rendericé la página real a cinco anchos y medí el resultado sobre los píxeles ya pintados: el ancho del menú se tomó hasta donde termina la franja verde oscuro, y el desbordamiento restando el ancho del documento al ancho de la ventana.

Ancho de ventana	Ancho real del menú	Zona de contenido	Desbordamiento horizontal	¿Cumple?
1440 px	240 px	1200 px	0 px	CUMPLE
1024 px	240 px	784 px	0 px	CUMPLE
900 px	240 px	660 px	0 px	CUMPLE
768 px	240 px	528 px	0 px	CUMPLE
520 px	240 px	280 px	0 px	CUMPLE

Punto 2 — Las cuatro tarjetas con auto-fit

Las cuatro tarjetas de indicadores no llevan anchos fijos ni `float`. Con `repeat(auto-fit, minmax(220px, 1fr))` el navegador calcula cuántas tarjetas caben en la fila según el espacio que haya, y las reparte. Al angostar la ventana, las tarjetas se van pasando de cuatro en una fila a tres, luego dos y por último una, sin que yo escriba nada.

```
.indicadores {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

/* en ventanas angostas el minimo baja a 160 px para que
   quepan dos tarjetas y no queden en una sola columna
   con tanto espacio vacio */
@media (max-width: 40rem) {
  .indicadores { grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); }
}
```

Reacomodación medida en el navegador

Ancho de ventana	Tarjetas por fila	Filas que se forman	Reparto medido	¿Se reacomodó?
1440 px	4	1	4 tarjetas x 4	SÍ
1024 px	3	2	4 tarjetas x 3	SÍ
900 px	2	2	4 tarjetas x 2	SÍ
768 px	1	4	4 tarjetas x 1	SÍ
520 px	1	4	4 tarjetas x 1	SÍ

Las cuatro tarjetas siguen siendo las cuatro en todos los anchos: lo que cambia es cuántas caben en cada fila. Eso lo decide el navegador solo, que es justo lo que se busca al usar `auto-fit`.

Punto 3 — Barra superior fija con position: sticky

A la barra superior le puse `position: sticky` con `top: 0`, para que se quede pegada arriba cuando se baja por la página. Importante: no es lo mismo que `position: fixed`. `sticky` sigue ocupando su lugar en el flujo —el contenido sigue debajo— y solo se pega cuando el scroll lo pasa; `fixed` la saca del flujo y habría que `reserve` el espacio a mano. Además `sticky` queda dentro de la zona `barra` de la rejilla, que es lo coherente con el punto 1.

```
.panel_barra {
  grid-area: barra;
```

```

display: flex;
align-items: center;
justify-content: space-between;
gap: 1rem;
padding: 0.6rem 1.2rem;
background: var(--color-marca);
color: var(--color-text-inverse);
position: sticky;
top: 0;
z-index: 10;
}

```

Comprobación con la tabla larga

La tabla de 16 mesas hace que la página mida **1581 px** de alto contra una ventana de **700 px**, así que hay scroll de sobra para probar. Bajé **900 px** y medí dónde quedó la barra y si seguía a la vista:

Medición	Valor	Qué significa
Página desplazada	900 px	El scroll sí se aplicó, no fue una medición en falso.
<code>top</code> de la barra antes de bajar	0 px	La barra arranca pegada al borde superior.
<code>top</code> de la barra después de bajar	0 px	Se quedó en el mismo sitio: ahí está funcionando el <code>sticky</code> .
¿Seguía visible en pantalla?	Sí	No se salió de la ventana al desplazarse.
Alto de la barra	63 px	Ocupa la franja de arriba completa.

El dato que importa es el del `top`: si el `sticky` no funcionara, al bajar **900 px** la barra habría quedado en un `top` negativo de unos **-837 px**. Se queda en **0**, o sea que está pegada.

Punto 4 — ¿Cuándo usé Grid y cuándo Flexbox?

Esta es la parte del punto 4, escrita con mis palabras.

Usé Grid cuando el problema es la página completa

Usé **Grid** cuando lo que tengo que armar es la estructura general de la página, o sea decidir quién ocupa qué lugar en filas y columnas al mismo tiempo. En el tablero me pasó justo eso: la barra arriba ocupará las dos columnas, el menú abajo a la izquierda, el contenido abajo a la derecha y el pie cerrando. Cada bloque tiene que ocupar una casilla distinta en las dos direcciones, y eso es exactamente lo que resuelve Grid.

Lo otro que me hizo decidirme por Grid fue que el menú tiene que medir siempre **240 px**. Con Grid declaro `grid-template-columns: 240px minmax(0, 1fr)` y queda dicho una sola vez: el menú mide 240 y lo que sobra es para el contenido. Si maquetara esto con Flexbox tendría que poner el ancho a mano y calcular con `calc()` el espacio que queda, y en cuanto algún elemento se pasara de largo, el cálculo se rompería. Grid separa la regla de la regla y se hace cargo del reparto.

También usé Grid para las cuatro tarjetas, pero con una función que ya es de Grid y no de Flexbox: `repeat(auto-fit, minmax(220px, 1fr))`. Esa línea dice "pon tantas columnas como quepan, pero que ninguna baje de 220 px", y el navegador lo recalcula solo cada vez que cambia el ancho de la ventana. Con Flexbox no tengo esa forma de pedirlo: allí la columna es una línea y para repartirlas tendría que calcular el ancho de cada tarjeta a mano, que es justamente el trabajo que quiero delegarle al navegador.

Usé Flexbox cuando el problema es una fila o una columna de cosas

Usé **Flexbox** cuando lo que tengo que repartir son unos cuantos elementos *dentro* de un espacio que ya está decidido, y solo me importa una dimensión: que queden en fila o en columna. Es el caso de la barra superior: el logo a la izquierda y el dato de la sesión a la derecha, en la misma fila y con el espacio repartido entre los dos.

Ahí Flexbox es más corto y hace exactamente lo que necesito:

```
.panel__barra {
  display: flex;
  align-items: center;
  justify-content: space-between; /* uno a cada extremo */
  gap: 1rem; /* separacion minima */
}
```

Con cuatro líneas queda resuelto. Si lo hubiera hecho con Grid tendría que declarar dos columnas, mandar el logo a una y la sesión a la otra, y alinearlos a mano, para terminar haciendo lo mismo con más código. También usé Flexbox en la lista de enlaces del menú: son tres elementos en vertical, y `flex-direction: column` los apila.

La regla que me quedó

Si el problema es “quién va arriba, quién va a la izquierda y quién ocupa el espacio que sobra”, uso Grid. Si el problema es “estos cuatro elementos en fila, uno al lado del otro, con el espacio repartido”, uso Flexbox.

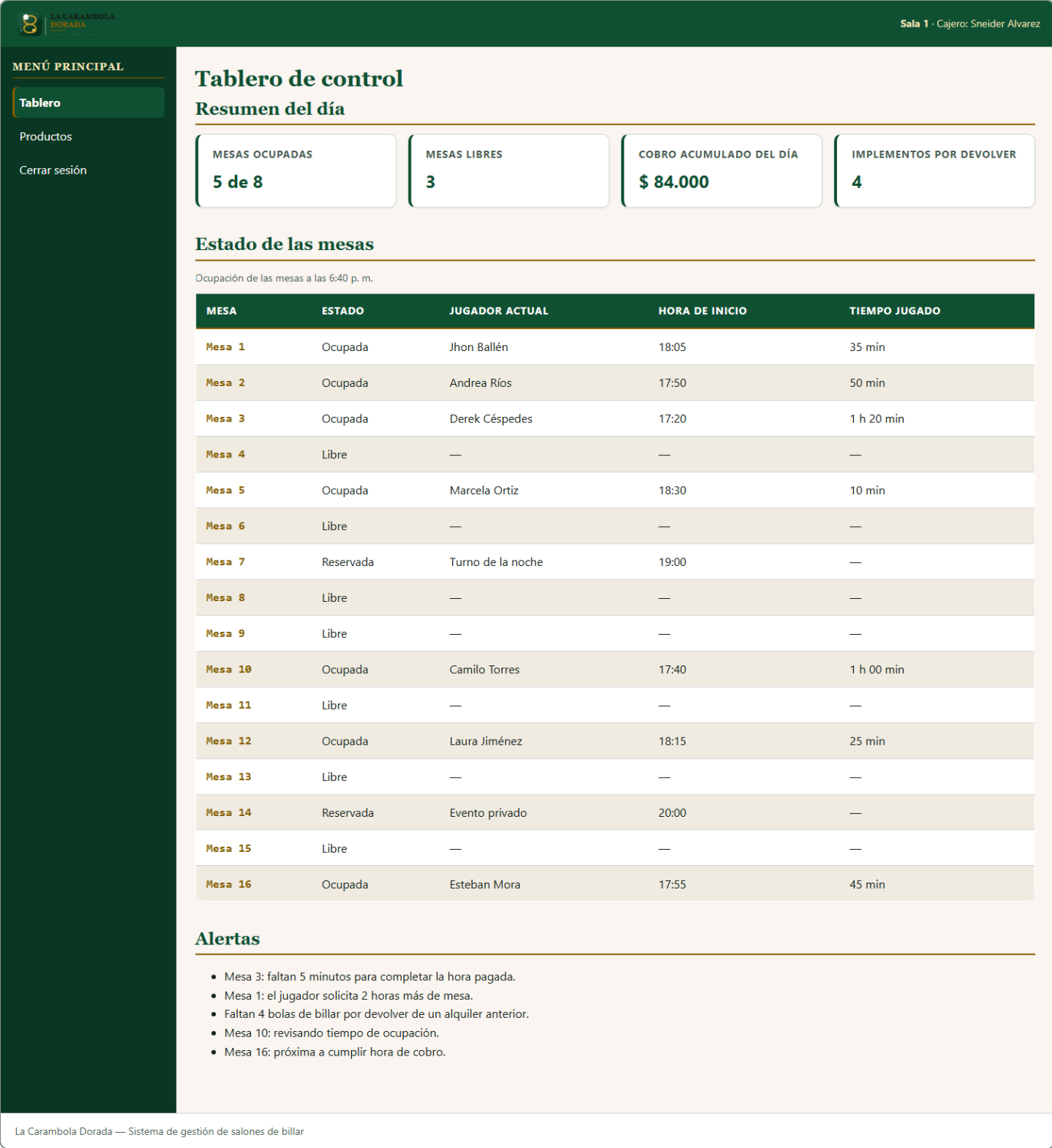
En este tablero usé Grid para las cuatro zonas de la página y para la fila de tarjetas, y Flexbox dentro de la barra y dentro del menú. No se mezcla por capricho: cada técnica se puso donde hace falta y sobra.

Criterios de evaluación

Criterio	Cómo se comprobó	Resultado	Medida
La página está maquetada con Grid y las zonas nombradas con <code>grid-template-areas</code> .	Leyendo la regla <code>.panel</code> en la hoja de estilos	CUMPLE	1 <code>grid-template-areas</code>
El menú lateral conserva su ancho de 240 px y el contenido ocupa el resto.	Midiendo el menú en el render, a cinco anchos	CUMPLE	240 px en los 5
El contenido no se desborda horizontalmente.	Restando el ancho del documento al de la ventana	CUMPLE	0 px en los 5
No se usa <code>float</code> ni <code>position: absolute</code> .	Buscando las dos propiedades en toda la hoja	CUMPLE	0 y 0
La barra superior queda fija al desplazarse.	Bajando 900 px y midiendo su posición	CUMPLE	<code>top</code> 0 → 0
Las tarjetas usan <code>repeat(auto-fit, minmax(...))</code> y se reacomodan.	Contando las tarjetas por fila a cinco anchos	CUMPLE	4 → 3 → 2 → 1
Sin hexadecimales sueltos y sin <code>!important</code> .	Revisando la hoja entera, sin contar los comentarios	CUMPLE	0 y 0
La página valida en el W3C.	Validador del W3C sobre <code>dashboard.html</code>	CUMPLE	0 errores

Evidencia del día

Las tres capturas se generaron renderizando el `dashboard.html` real con el motor del navegador, a tres anchos de ventana distintos, sin barra de direcciones.





MENÚ PRINCIPAL

Tablero

Productos

Cerrar sesión

Tablero de control

Resumen del día

MESAS OCUPADAS

5 de 8

MESAS LIBRES

3

COBRO ACUMULADO DEL DÍA

\$ 84.000

IMPLEMENTOS POR DEVOLVER

4

Estado de las mesas

Ocupación de las mesas a las 6:40 p. m.

MESA	ESTADO	JUGADOR ACTUAL	HORA DE INICIO	TIEMPO JUGADO
Mesa 1	Ocupada	Jhon Ballén	18:05	35 min
Mesa 2	Ocupada	Andrea Ríos	17:50	50 min
Mesa 3	Ocupada	Derek Céspedes	17:20	1 h 20 min
Mesa 4	Libre	—	—	—
Mesa 5	Ocupada	Marcela Ortiz	18:30	10 min
Mesa 6	Libre	—	—	—
Mesa 7	Reservada	Turno de la noche	19:00	—
Mesa 8	Libre	—	—	—
Mesa 9	Libre	—	—	—
Mesa 10	Ocupada	Camilo Torres	17:40	1 h 00 min
Mesa 11	Libre	—	—	—
Mesa 12	Ocupada	Laura Jiménez	18:15	25 min
Mesa	...	...	...	...



Figura 2. Ventana de 900 px. Las tarjetas ya se reacomodaron a dos por fila (4 tarjetas x 2), el menú sigue en 240 px y la tabla termina en 872 con límite 874.



MENÚ PRINCIPAL

Tablero

Productos

Cerrar sesión

Tablero de control

Resumen del día

MESAS OCUPADAS

5 de 8

MESAS LIBRES

3

COBRO ACUMULADO DEL
DÍA

\$ 84.000

IMPLEMENTOS POR
DEVOLVER

4

Estado de las mesas



Ocupación de las mesas a las 6:40 p. m.

MESA	ESTADO	JUGADOR ACTUAL	HORA DE INICIO
Mesa 1	Ocupada	Jhon Ballén	18:40
Mesa 2	Ocupada	Andrea Ríos	17:40
Mesa 3	Ocupada	Derek Céspedes	17:40
Mesa 4	Libre	—	—
Mesa 5	Ocupada	Marcela Ortiz	18:40
Mesa 6	Libre	—	—
Mesa 7	Reservada	Turno de la noche	19:00
Mesa 8	Libre	—	—
Mesa 9	Libre	—	—

Mesa 10	Ocupada	Camilo Torres	17
Mesa 11	Libre	—	—
Mesa 12	Ocupada	Laura Jiménez	18
Mesa 13	Libre	—	—
Mesa 14	Reservada	Evento privado	20
Mesa 15	Libre	—	—
Mesa 16	Ocupada	Esteban Mora	17

Alertas

- Mesa 3: faltan 5 minutos para completar la hora pagada.
- Mesa 1: el jugador solicita 2 horas más de mesa.
- Faltan 4 bolas de billar por devolver de un alquiler anterior.

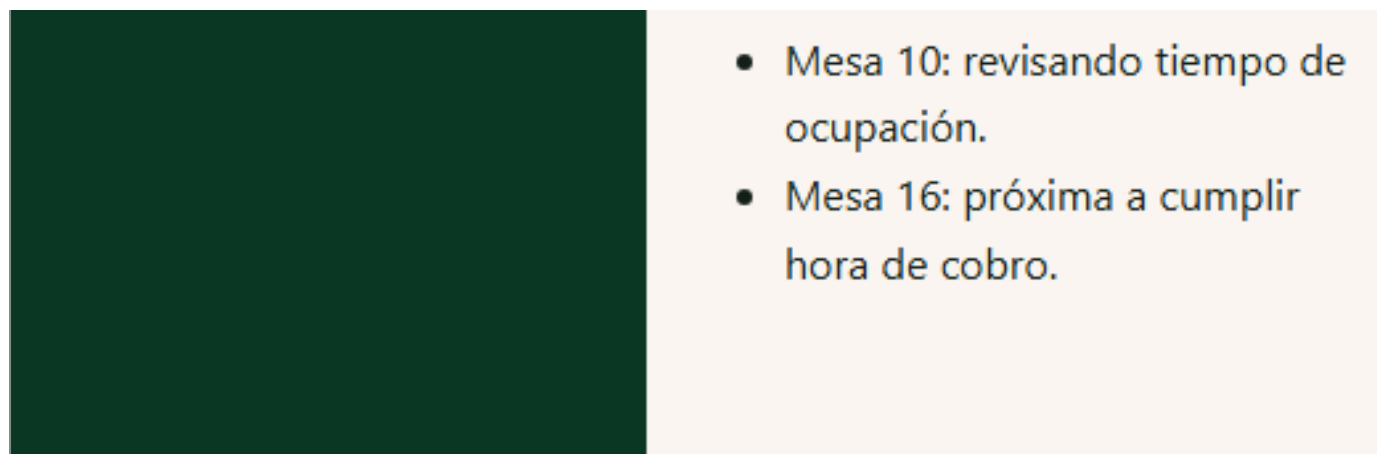


Figura 3. Ventana de **520 px**, la más angosta que se probó. Las tarjetas quedan en **una columna**, el menú **no se apiló ni se ocultó**: conserva sus 240 px como pide el enunciado. La tabla se desplaza con su propio scroll dentro del contenedor y la página no se desborda (**507** contra un límite de **508**).

Nota sobre cómo se midió: la navegador que tengo automáticamente ya no saca el código de la página, y su captura sale en blanco si la página queda desplazada. Para no quedarme con la medición a ojo, el script de medición escribía cada dato dentro del color de un bloque de la pantalla y yo leí ese color píxel por píxel. Todas las cifras de esta bitácora salen de ahí, no de suponer lo que debería pasar.

Sneider Alvarez Santos
Aprendiz — Ficha 3428392

Recepción / Instructor